

# Solution Officielle du TP : API Gateway Microservices

## Partie 1 : Objectif et Fonctionnement du Gateway

L'objectif de ce projet est de centraliser l'ensemble des requêtes sur un point d'entrée unique configuré sur le port 3000. Le client (Postman) communique uniquement avec la passerelle, qui se charge de transférer de manière transparente les requêtes vers les microservices concernés via axios sans modifier leur comportement d'origine.

## Partie 2 : Configuration pas-à-pas

### 1. Commandes d'initialisation

```
mkdir gateway
cd gateway
npm init -y
npm install express axios
npm install --save-dev nodemon
```

### 2. Fichier package.json

```
{
  "name": "api-gateway",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "nodemon index.js"
  },
  "dependencies": {
    "axios": "^1.6.0",
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  }
}
```

### 3. Fichier principal index.js (Version Conforme Standard)

```
const express = require("express");
const axios = require("axios");

const app = express();
app.use(express.json());

// Adresses de redirection des microservices existants
const LIVRE_SERVICE = 'http://127.0.0.1:3001';
const MEMBRE_SERVICE = 'http://127.0.0.1:3002';
const EMPRUNT_SERVICE = 'http://127.0.0.1:3003';

// =====
```

```

// ROUTAGE : SERVICE LIVRE (Port 3001)
// =====

app.get('/livres', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres`);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.get('/livres/:id', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres/${req.params.id}`);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.post('/livres', async (req, res) => {
  try {
    const r = await axios.post(`${LIVRE_SERVICE}/livres`, req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.put('/livres/:id', async (req, res) => {
  try {
    const r = await axios.put(`${LIVRE_SERVICE}/livres/${req.params.id}`, req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.patch('/livres/:id/disponibilite', async (req, res) => {
  try {
    const r = await axios.patch(`${LIVRE_SERVICE}/livres/${req.params.id}/
disponibilite`, req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.delete('/livres/:id', async (req, res) => {
  try {
    const r = await axios.delete(`${LIVRE_SERVICE}/livres/${req.params.id}`);
  }
});

```

```

        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

// =====
// ROUTAGE : SERVICE MEMBRE (Port 3002)
// =====

app.get('/membres/:id', async (req, res) => {
    try {
        const r = await axios.get(`${MEMBRE_SERVICE}/membres/${req.params.id}`);
        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.post('/membres', async (req, res) => {
    try {
        const r = await axios.post(`${MEMBRE_SERVICE}/membres`, req.body);
        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

// =====
// ROUTAGE : SERVICE EMPRUNT (Port 3003)
// =====

app.post('/emprunts', async (req, res) => {
    try {
        const r = await axios.post(`${EMPRUNT_SERVICE}/emprunts`, req.body);
        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.patch('/emprunts/:id/retour', async (req, res) => {
    try {
        const r = await axios.patch(`${EMPRUNT_SERVICE}/emprunts/${req.params.id}/retour`,
        req.body);
        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.listen(3000, () => {

```

```
    console.log('API Gateway standard active sur le port 3000');  
  });
```

### Partie 3 : Commandes d'exécution globales

Ouvrez un terminal pour chaque service et lancez-les simultanément :

- `cd livre-service && npm run start` (Port 3001)
- `cd membre-service && npm run start` (Port 3002)
- `cd emprunt-service && npm run start` (Port 3003)
- `cd gateway && npm run start` (Port 3000 — Unique point d'entrée client)